Dictamen independiente — MediaWorker C++ de GoldenBridge

Fecha: 2026-10-09. Encargo: `prompts/PROMPT_codex_mediaworker_cpp_escenarios_20261009.md`.

0. Dictamen y alcance

[INFERENCIA] **Ajustar las decisiones 1 y 3; aceptar C++ sobre C con condiciones.** El mecanismo correcto es retirar el coordinador compartido y el viaje al executor de la cadena frecuente de audio. El lenguaje no prueba la ganancia. **A debe ser el primer escenario nativo bajo prueba**, con I/O asíncrono en un reactor de un único hilo por llamada, Silero en línea y SmartTurn fuera del reactor. Antes de comprometer la reescritura, medir el monolito a 60 ms y una variante Python 3.14t con loop por llamada y DSP en línea. S con W=1 es un control causal obligatorio; B es un contraste posterior, condicionado por el ownership del WebSocket.

Decisión provisional	Dictamen	Motivo y condición
1. Reescribir Media en nativo; descartar Python	[INFERENCIA] **Ajustar**	El GC compartido es un riesgo real, pero sus pausas en este diseño no se midieron. Un loop Python por llamada también elimina la contención del loop global si llama al DSP síncrono directamente. Hacer un PoC acotado antes del port; C++ queda como candidato, no como necesidad demostrada.
2. C++ 17/20 en vez de C	[INFERENCIA] **Aceptar condicionado; preferir C++20**	RAII y ownership explícito ayudan. `jthread` y `stop_token` requieren C++20. ORT también tiene API C válida; el ahorro no proviene de llamar a ORT desde C++. Validar toolchain, artefacto ORT y soxr. No adoptar `ORT_NO_EXCEPTIONS` como recuperación por llamada.
3. A principal, S y B contrastes	[INFERENCIA] **Ajustar, conservando A primero**	"Síncrono" debe describir el DSP y la decisión, no esperas bloqueantes de red. SmartTurn no puede bloquear A. S no debe ejecutar inferencias largas en el shard. B no puede compartir libremente un stream Beast entre sus dos hilos.

[MEDICIÓN] El corpus consolidado de `CLAUDE.md`, "Hechos establecidos", reporta rodilla 16–25 sesiones a 20 ms en .143, ciclo p90 122 ms en 41+, Silero 401 µs/chunk y TTFAB p50/p90 558/1.330 ms. **Son mediciones previas aportadas**, no resultados obtenidos durante este dictamen. Se aplican las correcciones de ese archivo a `EVENT_LOOP.md`: 2.073,5 ms por ventana de 5 s; perf confundió Silero con SmartTurn; el spin era OpenBLAS/OpenMP; simultaneidad por barrido de altas/bajas.

[PENDIENTE] K a 60 ms en .143 y Modal, configuración efectiva, reservas CPU/memoria, wrapper Modal, límites del contenedor y rendimiento nativo. Ninguno se deduce del prompt ni de una biblioteca. No se ejecutaron pruebas de carga, gates, compilaciones ni sanitizers; este documento diseña esas verificaciones.

[CÓDIGO] Se resolvieron los objetos git: GoldenBridge `api` = `08932a64645bff74030ad2bd63543dd2334b8101`; ai-engine **dentro de este workspace**, `Asterisk-AI-Voice-Agent-FX`, rama indicada = `d059295d2468486bc6f8d2c883a6e41599d695f9`. GoldenBridge contiene cambios locales en config, servidor y cliente, además de archivos sin seguimiento; la evidencia productiva siguiente se leyó con `git show api:` y `git grep … api`, sin incorporarlos ni modificarlos. ai-engine no mostró cambios con `git status --short`.

[DOCUMENTACIÓN] Contexto consultado: `CLAUDE.md`, arquitectura autoritativa, auditoría H1–H8, dictámenes de multiproceso y frontera, README/matriz/propuesta/dictamen de `repo_analisis`, los ocho informes y `EVENT_LOOP.md`. Los informes de terceros son evidencia documental hasta contrastar su fragmento con el clon; no convierto sus conclusiones o comentarios en benchmarks. Los commits de terceros corresponden al manifiesto `repo_analisis/IDENTIDAD.tsv`.

[DOCUMENTACIÓN] Coordenadas abreviadas usadas en las citas: **WS** = GoldenBridge `src/server/websocket_server.py`; **SMART** = `src/processors/smart_turn_processor.py`; **VAD** = `src/processors/vad_processor.py`; **STENO** = `src/processors/steno_stt_processor.py`; **TTS** = `src/processors/tts_processor.py`; **CODEC** = `src/audio/codec.py`. Todas las líneas corresponden a `api@08932a6`. **ENG**, **DG**, **AS** = ai-engine `src/engine.py`, `src/providers/deepgram.py`, `src/audio/audiosocket_server.py` en `d059295`. Una cita abreviada identifica archivo, símbolo y líneas.

1. Pregunta 1 — tesis y costos que sobreviven

[CÓDIGO] La ruta productiva es `_session_processor` → `_run_audio_frames` → `_handle_audio_message` o `_handle_audio_batch` → `process_audio[_batch]` → executor `vad` → `_process_frame_sync` → retorno al loop → `_audio_post_vad`. El batch prepara cada mensaje, hace un único viaje, procesa posts en orden y devuelve el resto a carry sin repetir el pre: WS: `_run_audio_frames/_handle_audio_batch`:2789–2968; SMART: `process_audio/process_audio_batch`:1046–1236. El pool global tiene cuatro workers por defecto: VAD: `_vad_executor`:38–73.

[CÓDIGO] El batch se deshabilita mientras TTS está vivo, nace, reproduce o hay conexión STT pendiente: WS: `_batch_eligible`:2719–2753. Por ello, a 60 ms sigue habiendo viajes por **mensaje** durante TTS; un mensaje no equivale a un chunk Silero.

VAD: `process_audio_float_sync`:434–467 acumula muestras y consume chunks de 256 a 8 kHz, uno o dos por mensaje de 480 muestras según el residuo.

[INFERENCIA] Retirar executor y future elimina **ese** submit/resume y su cola compartida. No elimina esperar CPU, completar I/O, despachar timers, hacer DSP, copiar buffers ni serializar salidas. Un reactor sigue siendo un loop: en A queda uno por llamada; desaparece el cuello de un loop que coordina todas, no la noción de loop.

Parte medida o ejecutada	A	S	B
Submit frecuente al pool VAD	[INFERENCIA] Eliminado si Silero corre en el dueño	[INFERENCIA] Eliminado si Silero corre en el shard	[INFERENCIA] Eliminado si Silero corre en entrada
Resume de future VAD	[INFERENCIA] Eliminado; sobreviven completions locales de sockets	[INFERENCIA] Eliminado; sobrevive cola de callbacks del shard	[INFERENCIA] Eliminado; sobreviven completions y comunicación entre direcciones
Pre: contador, audio_buffer, adopción STT	[INFERENCIA] Trabajo permanece; adopción como estado async	[INFERENCIA] Igual, con presupuesto por callback	[INFERENCIA] En entrada; no compartir el buffer con salida
Job: codec, RMS, soxr, Silero, buffers	[INFERENCIA] Permanece; cambia asignación/copias, a medir	[INFERENCIA] Permanece y ocupa el shard	[INFERENCIA] Permanece en entrada
SmartTurn	[INFERENCIA] Submit/completion por comprobación al pool	[INFERENCIA] Igual, sin bloquear shard	[INFERENCIA] Preferir pool; hacerlo en entrada frena VAD
Post: STT, banking, barge-in, cierre	[INFERENCIA] Permanece como máquina de estados	[INFERENCIA] Permanece; ningún await de red bloqueante	[INFERENCIA] Entrada decide, salida confirma corte
Pacer y emisión	[INFERENCIA] Timer/write completion local	[INFERENCIA] Timer/write completion de shard	[INFERENCIA] En salida, con coordinación de socket
Observabilidad	[INFERENCIA] Registro local amortizado	[INFERENCIA] Registro por shard amortizado	[INFERENCIA] Registro por dirección amortizado

[CÓDIGO] El job contiene asignaciones que una migración literal conservaría: `np.concatenate` para pending_audio y turn_audio, copias del snapshot, conversiones/tobytes y RMS. SMART: `_process_frame_sync`:1614–1735,1785–1834; VAD: `process_audio_float_sync`:451–465; CODEC: `mulaw_to_pcm16/resample_audio`:81–100,269–296. Un ring preasignado puede reducirlas; **esa es una segunda hipótesis**, distinta de eliminar coordinación.

[CÓDIGO] El post intenta enviar a Steno **antes** de comprobar barge-in; si falla, espera `reconnect_session`: WS: `_audio_post_vad`:3780–3886. La reconexión incluye handshake/metadata y reemplazo de tareas: STENO: `reconnect_session`:929–988. Finalize espera resultados y sus timeouts: STENO: `_send_finalize_locked`:645–720. El defer por TTS puede esperar hasta 30 s: WS: `_handle_new_user_turn_tts`:8259–8281. Cambiar de lenguaje sin convertir estas esperas a estados puede atascar A o un shard entero.

[CÓDIGO] El CYCLE-PROBE mide el batch desde el comienzo del pre, no toda la residencia anterior en inbox. Divide `resume` en retorno VAD y SmartTurn: WS: `_handle_audio_batch`:2923–2959. Un percentil de ciclo por lote no es directamente el percentil trama→decisión de todos los mensajes.

[INFERENCIA] La tesis **ataca la causa medida** si el socket termina en el dueño y la ruta frecuente no atraviesa otro coordinador. No prueba 100 llamadas, ni menor CPU. Puede trasladar el límite al scheduler, allocator/arena ORT, pool SmartTurn, callbacks largos, TLS o proveedor; hay que medir el máximo componente de la latencia y la rodilla completa.

2. Pregunta 2 — rehabilitar Python antes del port

[DOCUMENTACIÓN] CPython free-threaded puede suspender hilos con estado Python adjunto durante GC; no significa que toda ejecución C autónoma se pare de la misma manera. La documentación actual de la serie 3.14 debe contrastarse con el build fijo 3.14.7: estados de hilo. La documentación de free-threading explica además que extensiones incompatibles pueden habilitar el GIL.

[INFERENCIA] Descartar Python **por posibilidad** de GC/refcount fue prematuro. Cientos de hilos del SO también existen en C++; los campos de SessionState sólo se vuelven compartidos si el diseño los comparte. Con un loop por llamada, estado/timers/conexiones propios, el problema de dos dueños no aparece por cambiar el número de loops. Sí quedan modelos/extractor compartidos y runtime GC.

[CÓDIGO] Hay DSP síncrono reutilizable: SMART: `_process_frame_sync`:1589–1882 y VAD: `process_audio_float_sync`:434–467. `process_audio` conserva el offload y cae al legacy si se desactiva el pool: SMART:1065–1087; `create_processors` fuerza frame_offload off

con workers<=0: `src/main.py`:143–154. **Poner workers=0 no implementa la comparación deseada**.

[INFERENCIA] PoC P: un hilo con asyncio/uvloop propio por llamada, aceptación con traspaso único del socket, clientes **asyncio** Steno/Ursula en ese loop, DSP síncrono directo y SmartTurn al mismo pool acotado usado en A. No usar `websockets.sync` como sinónimo de "un hilo total": sus lectores auxiliares alterarían la contabilidad. Ningún lock, Queue, tarea o HTTP client ligado a un loop se comparte con otro. Sólo resultados de baja frecuencia vuelven al dueño.

Medición para rehabilitar P	Prueba y resultado exigido [INFERENCIA]
GIL efectivo	Leer `sys._is_gil_enabled()` tras importar todas las dependencias, no sólo bandera de instalación
Pausas GC	Correlacionar callbacks GC con lag de **cada** loop y schedstat; churn de alta/baja y heap representativo, además del mínimo sin Conversación
Refcount/shared resources	Profiling de locks/runtime; modelo/extractor compartidos frente a réplicas pequeñas, sin cambiar DSP ni tráfico
Costo SO	Hilos efectivos, runnable, cambios de contexto, runqueue wait, CPU y memoria a 100; 500 sólo stress exploratorio
Seguridad	Reusar racetests, ampliar lifecycle; no transferir objetos asyncio entre loops
Cadencia y orden	60 ms completos, parciales 20/40 ms, jitter y silencio; hashes de entrada/STT/salida
Resultado E2E	Mismos criterios §10 que A; conservar cabeza de interrupción, turnos y playback, no sólo mejorar resume
Coste del protocolo	Mismo Steno/Ursula/pacer y mismos límites que A; no usar un echo server como sustituto

[INFERENCIA] Presupuesto acotado: **5–8 días-persona** incluyendo instrumentación y campañas locales, con ingeniero familiarizado con GB y entorno 3.14.7 listo. Frente a **12–20 días-persona sólo de port DSP con paridad** (§11), es una opción barata que podría evitar el port o identificar exactamente su necesidad. Estos rangos no son mediciones. Si P cumple los criterios bajo churn, se rehabilita; si pausa GC/contención explica incumplimientos reproducibles, se justifica C++.

[PENDIENTE] Las pruebas t1–t8 existen en `tools/racetests`; no se ejecutaron aquí ni son prueba de seguridad del diseño nuevo. t3 no aparece como archivo independiente en el árbol api consultado. No presento ocho scripts como ocho gates ya pasados.

3. Pregunta 3 — C, C++ y bibliotecas

[DOCUMENTACIÓN] ORT ofrece API C y distribuciones nativas. Su header C++ 1.24.4, encabezado L4–21, es un wrapper con ownership/RAII sobre la API C. El número de inferencias Silero no hace intrínsecamente más rápida la interfaz C++.

[INFERENCIA] Prefiero C++20 por recursos con ownership move-only, buffers tipados, destrucción por etapas y cancelación cooperativa. C también admite cleanup estructurado y no exige copiar el sistema de refcounts de FreeSWITCH. C no "sólo gana" en módulos o ABI: experiencia del equipo, dependencias, tamaño/tiempo de build y simplicidad pueden inclinarlo. No hay razón local suficiente para reemplazar C++ por C.

[DOCUMENTACIÓN] **Corrección de excepciones:** el header C++ ORT 1.24.4 L37–73 hace `abort()` por defecto cuando se define `ORT_NO_EXCEPTIONS`; no transforma fallos en códigos recuperables. Fuente fijada.

[INFERENCIA] Mantener RAII; usar API C con `OrtStatus` en el borde de inferencia si se exige error explícito, o capturar excepciones en la frontera de la llamada. Ninguna excepción cruza ABI C ni escapa del hilo. "Cero asignaciones" debe ser un objetivo verificado del código propio **después de warm-up**, no una garantía sobre ORT/TLS/Beast. Instrumentar allocations por inferencia y p99 antes de optimizar.

[DOCUMENTACIÓN] Beast permite reactor propio y no crea hilos automáticamente; sus streams requieren serialización de acceso. Puede haber un async_read y un async_write pendientes, iniciados/gestionados dentro del mismo ejecutor serializado. No garantiza dos hilos operando libremente el objeto. Contrato Beast.

[DOCUMENTACIÓN] Las bibliotecas citadas no son equivalentes: IXWebSocket: `WebSocket::start` crea un `std::thread` por objeto, L167–172 de IXWebSocket.cpp; uWebSockets organiza sockets en el loop dueño y comunicación con `Loop::defer`: documentación del proyecto. Que estén escritas en C++ no entrega automáticamente A con tres sockets y un hilo total.

[INFERENCIA] Para A/S escoger primero Beast/Asio con operaciones de red asíncronas, cadena DSP síncrona y un ejecutor por dueño. No escribir un stack RFC6455/TLS directamente sobre epoll para esta prueba. Para B resolver el stream bidireccional antes de vender cero handoffs (§4).

Aspecto	Decisión o verificación
Toolchain	[DOCUMENTACIÓN] Trixie ofrece g++14: paquete oficial. [INFERENCIA] Build multi-stage con GCC/Clang, CMake/Ninja, headers Beast/OpenSSL/soxr y versiones fijadas; sólo runtime en final.
Imagen actual	[CÓDIGO] `Dockerfile`:42–79 instala base, libgomp, CA, uv e intérprete; no prepara un toolchain C++ completo. Pins de hilos en L104–109.
ORT nativo	[INFERENCIA] Artefacto CPU Linux x64 1.24.4 verificable y headers de esa versión; leer versión efectiva, provider, hashes y dependencias con readelf/ldd. Evitar enlazar por accidente una biblioteca privada del wheel.
Equivalencia wheel/nativo	[PENDIENTE] Mismo número 1.24.4 no prueba mismas opciones de build, kernels o provider. El gate con el binario realmente desplegado cierra este punto.
soxr	[CÓDIGO] `requirements-314t.lock`:104 fija **Python-soxr 1.1.0**, no una versión C llamada 1.1.0. [DOCUMENTACIÓN] libsoxr de Trixie es 0.1.3-4+b2 en amd64. [PENDIENTE] Identificar versión/build C que usa el wheel antes de sustituirlo.
TLS	[INFERENCIA] OpenSSL/CA/SNI/hostname verification, handshake async con timeout; medir handshakes simultáneos y steady-state aparte. No omitir validación para ganar latencia.
ABI	[INFERENCIA] ABI C opcional para empaquetar el core: handles opacos, ownership/documentación de buffers y códigos de error. No es el transporte entre procesos.
Tamaño/build	[PENDIENTE] Medir binario stripped, símbolos separados, capas comprimidas, link y compilación limpia/incremental CI. No recompilar ORT por cada cambio de Media; comparar artefacto precompilado frente a build propio sólo si lo exige paridad.

4. Pregunta 4 — tabla A/S/B corregida

[INFERENCIA] Las celdas siguientes son **diseño propuesto**, no capacidad observada. “Sin locks” se refiere al estado mutable dueño de llamada; hay recursos compartidos, queues de inferencia y métricas. “Aislamiento total” no existe en un mismo proceso: todos comparten CPU, allocator, modelos y dominio de fallo.

Fila	A — reactor/hilo por llamada	S — W shards	B — entrada y salida por llamada
Espera	async reads/writes sobre ai-engine, Steno, Ursula; un io_context; timers y mailbox	un reactor por shard; presupuesto de trabajo y fairness por llamada	dos ejecutores; necesita política segura para el WS ai-engine bidireccional
Dueño de estado	uno; completions del pool vuelven por mensaje	uno por shard; sesión fija al shard	entrada posee DSP/STT; salida posee TTS/pacer; generación/cancelación/snapshots coherentes
I/O lenta	no bloquea reactor; cola local acotada, deadline y cierre por llamada	no debe bloquear; limitar callbacks/copias por turno del reactor	separar direcciones ayuda, pero el writer lento sigue acumulando cola de su llamada
Latencia DSP	Silero bloquea brevemente callbacks de **esa** llamada; medir	Silero bloquea otras llamadas del shard; medir carga media y cola	Silero bloquea entrada; salida puede seguir; CPU compartida impide promesa temporal absoluta
SmartTurn	pool acotado; no esperar bloqueando io_context	pool obligatorio para prueba comparable	pool recomendado; inline sólo como variante que acepta congelar entrada
Barge-in	cancel local antes de red; descartar generación y cortar writer	igual, más espera de callbacks del shard	flag/generación más wakeup de salida, chequeo justo antes de write
Límite de corte	objetivo <= tick + lag del dueño + write ya iniciado; no garantía SO	igual, con lag del shard	objetivo <= tick + lag de salida + coordinación; "<=1 tick" sólo con supuestos medidos
Handoffs por audio	cero entre hilos propios en ruta frecuente; sí completions/callbacks	igual dentro de shard	con Beast, al menos uno si salida entrega bytes al ejecutor dueño del WS; una variante de transporte distinta debe demostrar acceso concurrente seguro
Hilos a N llamadas	N + P inferencia + auxiliares medidos	W + P + auxiliares	2N + P + auxiliares; bibliotecas pueden añadir más
Balanceo	asignar dueño al aceptar, sin balanceo entre llamadas dentro del worker	least-load local con reserva antes de admitir; lifetime affinity; medir skew	asignar par de dueños; sin migración de llamada
Capacidad	no hay K_loop compartido entre llamadas; K_worker pendiente	K_loop = capacidad sostenible de cada shard; K_worker depende de W y distribución	como A por proceso; no asumir mayor K por tener dos hilos
Error recuperable	cierre idempotente de llamada; crash nativo tumba worker entero	excepción contenida en sesión; un bug nativo tumba todos los shards	mismo dominio de fallo, más riesgo de UAF y deadlock
Verificación	ASan/UBSan y pruebas de concurrency del pool/lifecycle; TSan donde sea viable	mismas; fairness y slow-peer	TSan de código instrumentable, ASan/UBSan y pruebas deterministas de interleavings obligatorias
Precedente	FreeSWITCH es analogía de ownership, no de ORT/WS/TLS	mediasoup es analogía de reactor sin IA	LiveKit SIP tiene goroutines M:N y colas; no equivale a 200 pthreads ni a compartir Beast
Riesgo principal	scheduler, allocations y callback largo en llamada	head-of-line por CPU; balanceo/fairness y cola local	ownership de socket, estados publicados incoherentes y destrucción con dos hilos
SIGTERM/drenaje	supervisor deja de admitir, señala a dueños, espera deadline y cierra	detener admisión; drain por shard con plazo común	detener admisión; parar ambos dueños, despertar waits y esperar a ambos antes de liberar
max_inputs	número de conexiones externas de la tarea, no número de hilos	igual; no reparte inputs entre shards	igual; dos hilos no consumen dos inputs por llamada
Watchdog 50 ms	timer dentro del dueño; gate de no-arrival y trabajo pendiente	timer por sesión en shard; repartir expiraciones	timer en entrada; lee snapshot de playback publicado por salida
Memoria	N pilas más estado de sesiones/colas/modelos	W pilas más el mismo estado de N sesiones	2N pilas y estado/colas de coordinación adicional

Fila	A — reactor/hilo por llamada	S — W shards	B — entrada y salida por llamada
Cancel/close durante inferencia	invalidar token; completion tardío no muta sesión cerrada	igual, sin bloqueo del shard en join	igual más confirmación de salida y retiro de buffer/stream

[CÓDIGO] Precedentes contrastados directamente: mediasoup `worker/src/DepLibUV.cpp:DepLibUV::RunLoop:10-11,110-120`; FreeSWITCH `src/switch_ivr_async.c:speech_callback:5185-5197`; LiveKit SIP `pkg/sip/media_pipeline.go:rtpLoop/rtpReadLoop:321-355`; LiveKit server `pkg/sfu/receiver_base.go:restartInternal/forwardRTP:310-340,928-968`. No contienen en esos fragmentos el pipeline de modelos de Golden; no extrapolo llamadas/núcleo.

[INFERENCIA] **B necesita una corrección estructural.** Con Beast, elegir un dueño del stream ai-engine —por ejemplo entrada— y recibir de salida bloques en una SPSC acotada, iniciando async_write en el dueño. Eso conserva aislamiento del cómputo de salida, pero no independencia total del timer/write frente a entrada ni "cero handoffs". Alternativa: un strand con ejecutor multihilo y estados sincronizados; deja de ser dos reactores con sockets disjuntos. Un mutex mantenido durante un recv bloqueante no sirve: impediría escribir. No aprobar B hasta demostrar el contrato de transporte y contar los handoffs reales.

Pilas, memoria y capacidad

[INFERENCIA] Ejemplos aritméticos, **no RSS medido**: si la pila reservada por hilo es s MiB, A reserva N×s y B 2N×s; S W×s. Auxiliares/pool se suman en las tres. Una reserva virtual no equivale a páginas residentes ni al límite físico del cgroup.

N	A con s=1 / s=8 MiB	B con s=1 / s=8 MiB	S con W=8, s=1 / s=8 MiB
100	[INFERENCIA] 100 / 800 MiB virtuales	[INFERENCIA] 200 / 1.600 MiB	[INFERENCIA] 8 / 64 MiB
500	[INFERENCIA] 500 / 4.000 MiB	[INFERENCIA] 1.000 / 8.000 MiB	[INFERENCIA] 8 / 64 MiB

[PENDIENTE] Leer tamaño efectivo con pthread_getattr_np, límites de stack/pids y mapas de memoria; medir páginas tocadas bajo carga y sanitizers. Reducir stack sólo tras high-water/canaries y stress; std::jthread no ofrece un parámetro portable de tamaño de pila.

[INFERENCIA] Definiciones obligatorias: **K_loop** = llamadas sostenibles por un loop Media bajo el SLO; **K_worker** = llamadas por proceso Media; **K_task** = suma efectiva de workers, limitada por CPU/memoria/desbalance; **K_flota** = capacidad agregada de tareas Modal. En A cada reactor sirve una llamada por diseño; no hay K_loop global para N. En S, W×K_loop es agregación, nunca aumento de K por loop. Incluso un S W=1 nativo puede mejorar eficiencia frente al Python actual; hay que medirlo, no prohibirlo por definición.

Watchdog, drenaje y complejidad

[CÓDIGO] WS: `_turn_close_watchdog`:8918–8987 duerme 50 ms, excluye TTS, exige 150 ms sin llegada, inbox/carry vacíos y cero frames in flight. SMART: `check_watchdog_close`:1970–2015 copia una huella y descarta el resultado si intercaló una trama. No se puede convertirlo en "si no llega audio, finalizar" ni en un timer que cierre mientras haya cola pendiente.

[INFERENCIA] Mantener esos gates con timers monotónicos y revisiones de estado. Un timer por sesión son aproximadamente 2.000 evaluaciones/s a 100 y 10.000 a 500 si despierta cada 50 ms; no son 2.000 hilos. Después de demostrar paridad puede usarse próximo deadline, con semántica de muestreo caracterizada. Timer, completion y frame deben deduplicar el cierre comprometido.

[INFERENCIA] SIGTERM: el handler sólo señala/despierta; un supervisor cierra listener/admisión, conserva existentes hasta plazo, invalida generaciones, cancela I/O y jobs, espera completions y destruye recursos. No hacer join desde el mismo hilo que debe completar el cierre ni desde el callback de una llamada reteniendo mutex. Cero zombies/FDs tras ambos estados: cierre normal y deadline agotado.

[PENDIENTE] La gracia de Modal para WS vivos durante redeploy/scale-down y el reenvío de señales al subprocess deben probarse en el wrapper real. No importar los 30 s documentados para el producto Modal Servers al decorador web_server sin evidencia.

[INFERENCIA] **B agrega más complejidad de corrección; S agrega más riesgo de interferencia entre llamadas.** A tiene menos estados concurrentes, pero más hilos que S. Mantengo A como principal nativo, S W=1 como control inmediato y S W>1 antes de B si la presión observada es CPU/scheduler. B sólo primero si una medición demuestra que el pacer pierde deadlines por cómputo de entrada aun con SmartTurn afuera.

5. Pregunta 5 — ORT compartido, lock y pool

[CÓDIGO] SMART: `create_session_instance`:133–151 comparte sesión, extractor y lock global. `predict_turn_complete`:205–225 **extrae features antes de tomar el lock**; éste cubre `session.run`, no todo SmartTurn. Silero comparte la sesión con wrappers/estado temporal aislados: VAD: `create_session_instance`:189–231. Hay una contradicción interna entre el comentario "ONNX not thread-safe" de SMART:145 y el "stateless, thread-safe Run" de VAD:193–214; comentarios no son contrato.

[DOCUMENTACIÓN] ORT **1.24.4**, `onnxruntime/core/session/inference_session.h:InferenceSession::Run:351-362`, autoriza invocaciones desde varios hilos sobre un modelo cargado e inicializado; los inputs y outputs del cliente no deben mutar durante la ejecución. Fuente versionada. Esto refuta la prohibición universal del comentario; no demuestra que buffers compartidos o cualquier provider/custom op sean seguros.

[INFERENCIA] Propuesta común: una sesión Silero inicializada y precalentada, LSTM/context/pending por llamada, tensors/buffers/RunOptions privados por operación; nunca dos inferencias de **la misma** llamada solapadas. Para SmartTurn, pool explícito pequeño y acotado, inicialmente P=2 como candidato experimental, comparado con P=1 manteniendo las demás variables. Features/scratch privados por job; datos copiados o con lifetime retenido; una comprobación en vuelo por llamada.

[INFERENCIA] Verificar `Run` concurrente CPU con outputs contra ejecución serial, incluidas decisiones marginales, primero con datos fijos y después con llamadas intercaladas. No crear/destruir sesión mientras corre Run ni compartir IoBinding mutable. Si esa variante pasa, quitar la serialización global **en el laboratorio**. Si no pasa o la arena contiende, comparar dos sesiones SmartTurn, una por hilo del pool; no 100 sesiones del modelo por defecto.

Escenario	Plan ORT/pool [INFERENCIA]
A	Silero en dueño; job SmartTurn con snapshot + token de revisión; completion vuelve al reactor
S	Silero en shard; SmartTurn y extracción/resampleo de su snapshot fuera del shard; misma política global P
B	Silero en entrada; SmartTurn al pool; salida recibe sólo cambios de generación/playback, jamás estado LSTM
Todos	Cuotas por sesión, límite global de jobs, expiración y rechazo local; no mutex retenido mientras se espera un resultado; sesión ORT vive hasta terminar todos los jobs

[MEDICIÓN] La tasa 0,0704 comprobaciones/s-sesión y CPU SmartTurn 0,0043 núcleos/llamada pertenecen al corpus de `CLAUDE.md`. [INFERENCIA] Su cociente aproxima **61 ms CPU por comprobación completa**. A 100, λ≈7,04/s y λ×0,061≈0,43 núcleos. **No es una medición de 43% de ocupación del lock**: features están fuera y CPU tampoco equivale necesariamente a wall time de Run. Tampoco demuestra cuándo empieza a encolar.

[INFERENCIA] Bajo la hipótesis adicional de 61 ms wall serializados por job, la utilización de un servidor sería 0,43 a 100 y 2,15 a 500. En el segundo caso no habría estabilidad de esa cola con un único servidor. Una ráfaga simultánea de diez jobs ocuparía aproximadamente 610 ms de servicio serial. Son modelos aritméticos para diseñar el stress, no capacidad demostrada del worker.

[PENDIENTE] Instrumentar separado: preparación/resampleo, log-mel, espera de queue, espera de lock, Run wall/CPU, completion y commit; medir p99 en llegadas naturales y ráfagas. Eso decide P, si necesita réplicas de sesión y el headroom de admisión.

[DOCUMENTACIÓN] IoBinding permite elegir/bindear memoria de entrada y salida; su motivación incluye evitar transferencias de dispositivo. [INFERENCIA] En CPU no elimina la coordinación ni hace thread-safe un binding compartido. Empezar con tensors sobre buffers preasignados y lifetime explícito; adoptar binding por job o por hilo sólo si un perfil demuestra ahorro y pasa paridad.

Memoria de modelos y señal

[INFERENCIA] Arrays teóricos: turn_audio de ocho segundos a 8 kHz float32 = **250 KiB** por sesión; ventana SmartTurn 8×16.000 float32 = **500 KiB**; features 80×800 float32 = **250 KiB** por job. A 100/500 sesiones, sólo esos turn_audio completos representan 24,4/122,1 MiB. Ventanas/snapshots/features y FFT pueden existir en varias copias; acotarlas a P jobs en vuelo evita multiplicarlas innecesariamente por N.

[CÓDIGO] Base de tamaños: SMART: `_process_frame_sync`:1808–1813 limita turn_audio a ocho segundos; `predict_turn_complete`:205–221 utiliza ventana 8×16.000; `src/processors/whisper_features.py:WhisperFeatureExtractor.__call__:89-105` produce mel80 con hop160 y descarta el último frame.

[PENDIENTE] RSS de cada modelo, pesos prepacked, arenas, workspace ORT y buffers TLS/WS. No hay tamaños de los artefactos ONNX en las rutas locales `GoldenBridge/smart-turn` y `GoldenBridge/modelos` inspeccionadas; no inferir RAM del modelo desde el nombre. Medir incrementos PSS/RSS al cargar 1/2/4 sesiones, después de warm-up y pico concurrente. Presupuesto: M_worker = modelos/arenas + N×estado/colas + P×scratch + pilas residentes + transporte; medir pendiente y residuo tras cerrar.

6. Pregunta 6 — compuerta de paridad del port

[CÓDIGO] El gate existente **no exige features bit-idénticas**: `test/test_reference_features.py:TOL_FEATURES/TOL_PROB_MARGINAL:48-51` fija 1e-3 y 0,05; `test_fidelidad_de_features`:170–184 usa max|delta|; `test_decision_estable_en_casos_confiados/marginales`:187–227 prohíbe volteos y acota drift marginal. `stack.json`:20–38 fija hashes, umbral 0,6 y catorce probabilidades. Los 1e-5 del shim contra transformers son otra comparación, no el umbral de arranque.

[CÓDIGO] Silero: `test/test_reference_vad.py:TOL_PROB`:39; `test_misma_cantidad_de_chunks/test_probabilidad_estable/test_transiciones_identicas/test_ningun_chunk_cambia_de_lado`:70–123 exige cantidad, max delta <1e-3, transiciones exactas y ningún cruce de umbral. La metadata está en **vad_stack.json**, además de vad_probs.npz; no alcanza stack.json para este gate.

[INFERENCIA] Compuerta propuesta en capas; falla cualquiera ⇒ no abrir listener. Comparar primero componentes aislados, después pipeline con timestamps virtuales y finalmente concurrencia. No regenerar NPZ para aceptar un port que decide distinto.

Capa	Criterio propuesto [INFERENCIA]	Evidencia/base
Identidad	Modelos/corpus/referencias por SHA256; shapes y dtypes; ORT 1.24.4 CPU, optimización/secuencial/intra1; ISA relevante	[CÓDIGO] `test_reference_features.py:test_corpus_intacto/test_mismo_juego_de_instrucciones:96-164`
Codec	256 códigos μ-law decode bit-idénticos; encode de todos los 65.536 valores PCM16 contra camino **efectivo**; little endian y límites explícitos	[CÓDIGO] CODEC: `mulaw_to_pcm16/pcm16_to_mulaw`:81–140
soxr	Misma versión/build C, io_spec int16, calidad y runtime; número de muestras/drain exactos; meta inicial: PCM16 idéntico. Si difiere, investigar, no aceptar automáticamente ±1 LSB	[CÓDIGO] CODEC: `resample_audio`:269–296
Log-mel	Todas las shapes; max absoluto <1e-3 como gate existente; registrar objetivo diagnóstico <=1e-5 frente al shim fijado, RMSE y caso peor	[CÓDIGO] `whisper_features.py:WhisperFeatureExtractor:47-105`; tests anteriores
ORT con features congeladas	Las 14 decisiones iguales usando comparación estricta p>0,6; drift marginal <0,05. Meta adicional conservadora: max delta de probabilidades <=1e-5 en mismo stack/ISA, sujeto a investigación si falla	[CÓDIGO] SMART: `predict_turn_complete`:223–229; `stack.json`:22–38
Cadena features→ORT	Mismas decisiones y drift marginal permitido, independientemente de que features pasen	[CÓDIGO] `test_reference_features.py`:187–227
Silero	max delta <1e-3 por chunk; cero cruces en ambos umbrales y transiciones idénticas; mismo reset recurrente/contexto/orden	[CÓDIGO] `test_reference_vad.py`:70–123
Gates y timers	Misma secuencia con reloj virtual: min_tts_time, duración, energía, chunks consecutivos, sensibilidad post-TTS, echo, fallback y watchdog	[CÓDIGO] SMART: `turn_met_barge_in_gates/_process_frame_sync`:902–938,1682–1735,1815–1882
Cancelación/estado	Cero bytes viejos después de barrera de quiescencia; cabeza de interrupción enviada una vez, orden antes de Finalize; cierre único	[CÓDIGO] WS: `_cancel_active_tts`:8485–8525,8583–8617
Concurrencia	Serial vs concurrente sin diferencias de decisión, mezcla, use-after-free ni job que resucite una sesión	[INFERENCIA] Stress reproducible con IDs y seeds

[CÓDIGO] El shim normaliza en float32 antes del padding interno con epsilon 1e-7; Hann periódico, reflexión centrada, rfft400/hop160, power, mel Slaney, clip1e-10, log10 y piso max−8: `whisper_features.py:WhisperFeatureExtractor:47-105`. El caller ya trunca/rellena a ocho segundos antes de invocarlo: SMART: `predict_turn_complete`:205–217. Hay que reproducir **ambos niveles**; no cambiar a normalización del audio corto como si fuera el camino efectivo de producción.

[INFERENCIA] Rompen paridad: FFT y precisión intermedia distintas, acumulación/dot en diferente orden, FMA o fast-math, redondeo, filtro mel reconstruido distinto, Hann simétrica, padding reflect vs cero, normalizar antes/después de completar la ventana, dividir PCM16 por 32767 vs 32768, estado/remainder perdido, reset soxr por reconnect, drenaje prematuro, modelo/kernel distinto aun con igual versión ORT. Empezar sin fast-math; mantener coeficientes fijados; comparar capas para atribuir el primer delta.

[CÓDIGO] El runtime puede importar `audioop` y usarlo antes de las tablas: CODEC:18–23,91–100,113–140; el lock incluye `audioop-lts==0.2.2` :9. [PENDIENTE] Leer HAS_AUDIOOP efectivo en 3.14t: no asumir que el fallback numpy es el oráculo activo. En particular, caracterizar overflow/signo de int16 en los extremos antes de "corregir" la codificación durante el port.

[CÓDIGO] NoiseReduction: `reduce_noise` :103–120 salta sólo entradas de menos de 100 ms. [INFERENCIA] Excluirlo es compatible con el laboratorio que limita cada entrada efectiva a <=60 ms y verifica ese guard, **no una equivalencia universal**: mensajes grandes, repacketización o salidas soxr acumuladas pueden superar la ventana. [PENDIENTE] Contar invocaciones/tamaños efectivos; bloquear formatos no soportados o caracterizar ese caso antes de producción.

Hallazgo independiente: el gate de arranque puede ocultar fallos

[CÓDIGO] `docker/arranque.sh` :1,17 declara /bin/sh y `set -eu` ; L43–44 evalúa `python -m pytest … | tail -20` dentro de `if ! …` ; L67 anuncia éxito y L72 sirve. No activa pipefail ni conserva por separado el estado de pytest. Los chequeos de existencia L25–35 sólo enumeran parte de las referencias.

[INFERENCIA] En un shell con semántica normal de pipeline sin pipefail, el status observado es el de tail; un pytest fallido seguido de tail exitoso puede entrar por la rama "referencia OK". Es un defecto estático de la ruta de error; **no afirmo haberlo provocado en la imagen desplegada**.

[INFERENCIA] En la integración, guardar y comprobar explícitamente el código de pytest/gate nativo, mostrar después el resumen y exigir todos los artefactos, cero skips y cantidad de casos esperada. Inyectar modelo corrupto, NPZ ausente, ISA no apta y decisión alterada: **el puerto debe permanecer cerrado**. El gate Python por sí solo tampoco certifica el binario C++ ni otra libonnxruntime.

7. Pregunta 7 — proceso, ABI e IPC frente a nanobind

[INFERENCIA] **Primera entrega: proceso Media separado + UDS local versionado.** ABI C y protocolo IPC son cosas distintas: `extern "C"` regula llamadas y layout dentro de una dirección de memoria; no conecta dos procesos. El core puede exponer ABI C para tests/futuro binding, pero Media↔Conversación se comunica con framing y mensajes, sin punteros.

Opción	Ventaja [INFERENCIA]	Riesgo/coste [INFERENCIA]
Proceso nativo + UDS	DSP autónomo sin estado Python adjunto; crash de Conversación separado; observa memoria/lag por proceso; reinicio definido	serialización, colas, supervisión y vida del canal; muerte Media pierde sus llamadas; reiniciar Conversación exige política de estado
Extensión nanobind	menos framing/copia de eventos; montaje compacto	crash nativo mata Conversación; finalización del intérprete/lifetime; APIs Python desde hilos reintroducen coordinación; mismo dominio de señales
ABI C del core	interfaz pequeña con handles opacos y status; prueba serial separada del transporte	ownership y threading igualmente deben documentarse; ABI no evita carreras ni garantiza compatibilidad Python

[DOCUMENTACIÓN] Nanobind soporta free-threading mediante `FREE_THREADED` ; un binding debe optar por ello y proteger estado. Para Python 3.14t no asumir la ABI estable provisional de 3.15+. Documentación nanobind.

[INFERENCIA] Una extensión que corre C++ puro sin objetos/callbacks Python por trama puede reducir su exposición al GC; no queda automáticamente detenida por toda pausa Python. Sin embargo, eventos y finalización vuelven al intérprete, y el dominio de fallo sigue común. La separación de proceso simplifica probar el aislamiento requerido y su coste IPC es a frecuencia semántica.

[DOCUMENTACIÓN] El contrato base y C1–C7 proceden de `doctos/DICTAMEN_FRONTERA_MEDIA_CONVERSACION_20260928.md` §5 y `repo_analisis/MATRIZ_COMPARATIVA.md` §3. [INFERENCIA] Conservar clave completa `(session_id, session_epoch, turn_id, speech_id, seq)` , versión, command_id/event_id y expiración. Audio sólo ai-engine↔Media y proveedores↔Media; Conversación sólo texto/eventos. Los siete ajustes quedan concretados así:

Corrección	Aplicación en proceso e in-process [INFERENCIA]
C1 cancel prioritario	Ingreso acotado; ejecutar cancel/close antes de datos admitidos pendientes. Cancel local no espera al IPC ni a C. Reservar capacidad de control.
C2 purga por clase	Invalidar/purgar SpeakChunk de speech_id; conservar ConfigureSession, funciones y cierre. Todo descartado queda contado.
C3 emitted_ms	Contar muestras/bytes **con write completado** por speech; separar emitido, aceptado por engine y reproducción estimada. played_ms requiere feedback, no llamarlo medido por un reloj de pared.
C4 parada en dos fases	Invalidar generación → detener productores/esperar callbacks viejos con deadline → reset/purga → abrir nueva. Cancel inmediata y liberación diferida son operaciones distintas.
C5 Accepted/Done/caducidad	Accepted confirma admisión, no ejecución ni transferencia real; Done/Error confirma resultado concreto. Timeout produce estado terminal; tardíos no ejecutan efectos. Retried function call no implica exactly-once externo.
C6 heartbeat con lag	Snapshot de lag máximo y percentiles por dueño/shard, queues/jobs/sesiones; proceso responde pero también reporta dueño congelado. Admisión usa lag/headroom; heartbeat no se fabrica en un hilo sano ignorando media atascada.
C7 seq	Recibir, validar y deduplicar en orden; hueco real cierra sesión. Prioridad de ejecución no cambia el orden de validación del transporte.

[INFERENCIA] **Conflicto a resolver: prioridad y seq.** No asignar seq cuando se encola un chunk y luego adelantar Cancel con un seq mayor: parecería pérdida, o descartaría comandos válidos si last_seq avanza. Opción mínima: asignar seq al serializar en orden wire **después** de elegir prioridad; recepción valida wire, y la cola de ejecución puede cancelar chunks ya validados. Chunks purgados antes del wire no consumen seq. Mantener event_id/command_id estables para dedup y dependencias explícitas para operaciones adelantadas. Limitar bytes ya escritos: un solo stream no puede adelantar cancel a bytes que ya están en TCP.

[INFERENCIA] Si la latencia del IPC acotado no cumple, separar físicamente control prioritario y datos con seq **por carril**, y barreras por generación/command_id entre carriles. Eso cambia el envelope/contrato y requiere pruebas; no conservar fingidamente un seq global FIFO mientras se reordena.

[INFERENCIA] `SpeechInterrupted` debe llegar/aplicarse en Conversación antes de `TurnCompleted` del interruptor; incluir referencia de dependencia y retener el siguiente turno si falta. Prioridad no basta para esta causalidad. Sólo el dueño confirma `SessionClosed` tras quiescencia; epoch anterior, turn viejo, speech cancelado y function response caducada nunca mutan el estado nuevo.

[CÓDIGO] Riesgo adicional al hacer SmartTurn/Finalize asíncronos: STENO: `discard_pending`:836–848 justifica purgar porque el receive-loop estuvo bloqueado y ningún audio nuevo llegó a Steno. [INFERENCIA] En el reactor nuevo esa premisa deja de cumplirse. Introducir barrera de turno STT: cerrar el segmento, mantener audio siguiente en cola separada o etiquetar respuestas si el proveedor lo soporta, y no purgar resultados válidos del siguiente turno. No inventar IDs que Steno no emite. Igual para Ursula: su flush está deshabilitado por defecto, TTS: `flush_tts_session`:576–588; una respuesta binaria tardía sin id no se distingue mágicamente. Drenar hasta frontera confirmada o reemplazar conexión antes de asociar audio a nueva generación.

8. Pregunta 8 — Modal y arranque

[DOCUMENTACIÓN] `@modal.web_server` expone un servidor propio en el puerto del contenedor y admite otros lenguajes: referencia Modal. [INFERENCIA] El wrapper Python puede lanzar el supervisor y terminar su bootstrap; no recibe ni relaya cada trama. Media C++ liga 0.0.0.0:8765 y acepta el WS proxied por Modal; Python Conversación usa UDS interno.

[DOCUMENTACIÓN] Modal soporta WS para web_server, mantiene una function call por conexión, admite concurrent inputs y no ofrece permessage-deflate: guía Web Functions. [INFERENCIA] max_inputs es el límite agregado externo por tarea; no configura hilos, shards ni capacidad de C. Si se expone un endpoint adicional C como función externa, sus inputs se contarían aparte; el UDS interno evita ese segundo endpoint.

[PENDIENTE] La búsqueda en api de `modal.web_server/modal.asgi_app` en src/tools/docker no produjo wrapper de GB. No se auditó el wrapper desplegado ni active.yaml; el default y el decorador hipotético no prueban el despliegue efectivo. Conseguirlos, leer max_inputs/target_inputs/cpu/memory/timeouts y medir salud de tarea antes de dimensionar.

[INFERENCIA] Secuencia de arranque: supervisor carga configuración efectiva y artefactos → ejecuta gates Python y nativo preservando status → Media inicializa los modelos y pasa el mismo gate con la biblioteca **de servicio** → Conversación 3.14.7t confirma handshake de contrato → listener listo/admisión. Una comprobación exitosa en otro proceso o con wheel diferente no cubre al core nativo. `startup_timeout` debe dar tiempo al bootstrap medido; no abrir puerto prematuramente para satisfacerlo.

[CÓDIGO] El arranque actual hace `exec python -m src.main` después del gate: `docker/arranque.sh`:70–72. [INFERENCIA] La integración requiere supervisor que propague señales, recoja hijos y publique readiness; no son dos procesos lanzados sin lifecycle común. Salud sin capacidad no debe admitir llamadas; max_inputs se fija desde K_task sostenible con margen, no desde el objetivo 100.

[INFERENCIA] Conversación puede omitir torch/ORT en su proceso, aunque conservarlos para el gate Python en la imagen inicial es más sencillo. Media nativo agrega libstdc++/OpenSSL/ORT/soxr y sus arenas; puede retirar el wrapper torch de su heap. No prometo menor memoria **total**: habrá dos heaps, IPC, pilas y quizá doble carga temporal de modelos durante el gate.

[MEDICIÓN] Los 33 s frío/0,3 s warm son antecedentes de CLAUDE.md para el despliegue anterior. [PENDIENTE] Medir por etapa descarga/capas, gate, modelos, handshake, listener y primer audio; p50/p90 en arranques independientes. Compilar durante construcción de imagen, no al arrancar. No afirmar que C++ reduce el frío sin medir esa cadena.

9. Pregunta 9 — riesgos y medición que los cierra

[INFERENCIA] Los riesgos siguientes son hipótesis concretas o consecuencias del diseño. Las pruebas propuestas no se realizaron en este encargo.

Riesgo	Medición/prueba de cierre
Scheduler con 100–1.000 hilos	[INFERENCIA] N=100 y stress N=500; schedstat por tid, runnable, context switches, CPU total/per-thread, lag de timers y throttling cgroup; leer cuotas/cpuset efectivos
Prioridad de hilos	[INFERENCIA] Mantener política normal, registrar política/nice efectivos; no reabrir SCHED_FIFO/RR/afinidad descartados. Correlacionar runqueue wait con deadline incumplido
Pilas/FD/pids	[INFERENCIA] stack reservado/residente/high-water, RLIMIT_NOFILE y pids.max; conteo FD real por llamada y tras churn. Tres sockets persistentes implican como base ~300/~1.500 a N=100/500, más reactors, wakeups, IPC y HTTP/fallback
SO_RCVBUF/SO_SNDBUF y TCP	[INFERENCIA] Leer getsockopt efectivo/autotuning, ocupación TCP, edad de bytes y retraso de reads; contar colas de kernel además del inbox. Reducir buffer puede trasladar presión al engine
TLS y alta simultánea	[INFERENCIA] Campaña TLS ON/OFF sólo diagnóstica local, nunca aceptación final sin TLS de proveedores; handshake CPU, cert/SNI, memoria por conexión, reconnect storms y p99
Tres TLS por llamada	[PENDIENTE] No asumir tres dentro del proceso: el tramo externo engine↔Modal tiene TLS, pero el hop proxy↔puerto Media y TLS saliente real dependen del wrapper/URLs. Leer topología efectiva y medir sólo terminaciones que posee Media
ORT/arena compartida	[INFERENCIA] Concurrencia serial/paralela determinista, allocations, locks y RSS después de warm-up; 1/2/4 sesiones bajo misma carga
Ráfaga SmartTurn	[INFERENCIA] Sincronizar cierres y mantener caso natural; queue wait vs Run; cuotas y expiración; ningún callback largo se ejecuta en dueño
I/O lenta en S	[INFERENCIA] Una llamada deja de leer/escribir; verificar p99 de otras del shard y resto; callback budget y queues en ms, no sólo mensajes
B: socket compartido/estado	[INFERENCIA] Interleavings deterministas y TSan; cancel durante write/read/close, snapshot coherente de TTS y generación, sin mutex que bloquee la otra dirección
Reconexión Steno/Ursula	[INFERENCIA] EOF, caída a mitad de Finalize/síntesis, metadata ausente y respuesta vieja tras nueva conexión; timeouts y quiescencia; comprobar política sin prometer replay del proveedor
Falta reconexión engine→GB	[CÓDIGO] DG:bloque de cierre de recepción:1762–1804 no inicia reconexión; ENG: `on_provider_event` :8201–8203 exige call_id. [INFERENCIA] Kill de WS debe producir recuperación definida o cierre de llamada; prerequisito operativo, no resultado del port
Barge-in oído vs emitido	[CÓDIGO] AS: `send_audio` :263–267 escribe/drain; no retira esos bytes. [INFERENCIA] Capturar Media write, bytes en engine, AudioSocket y audio recibido; sin flush externo no prometer corte audible de un tick
Cadencia y algoritmos temporales	[INFERENCIA] 20/40/60 ms parciales, coalescer stall, pausas sin frames y mensajes grandes; comparar decisions con reloj virtual y real. No sintetizar silencios sin caracterizar semántica
Cola sin tope y pérdida muda	[CÓDIGO] WS: `_PacedAudioSender.__init__` :614 crea cola ilimitada; config/default.yaml:46 inbox ilimitado. [INFERENCIA] Cotas por bytes/ms con política explícita; overflow de audio bruto cierra/degrada de forma definida, no descarta muestras arbitrariamente en prueba de paridad
Finalize mientras sigue entrando audio	[CÓDIGO] STENO: `discard_pending` :836–848 presupone entrada bloqueada. [INFERENCIA] Prueba dos turnos seguidos con Finalize tardío; barrera entre segmentos, cero pérdida/mezcla
Backpressure IPC y causalidad	[INFERENCIA] 50 SpeakChunk + cancel, funciones pendientes, drop/duplicate/reorder/seq gap, C pausada 2 s; orden interruption→turn sin esperar C para el corte
Lifetime/errores nativos	[INFERENCIA] ASan/UBSan, job que finaliza tras close, cancel doble, destructor con completions; errores de ORT tratados como fallo de llamada, no abort voluntario del worker
Drenaje Modal	[INFERENCIA] SIGTERM, redeploy, scale-down y deadline con WS vivos; cerrar admisión primero, señales a hijos, métricas de pérdida y cero zombies. Muerte abrupta del proceso sí pierde todas sus llamadas
Gate falso positivo	[INFERENCIA] Corromper referencias/modelos y retirar ISA en laboratorio; fallo no publica listener ni health listo; cero skips; validar código de salida
Observabilidad que crea cuello	[INFERENCIA] Rings por dueño, drenaje agregado; medir overhead ON/OFF. Sin log ni Prometheus labels bajo mutex global por cada trama
Negocio y semáforo TTS	[CÓDIGO] TTS: `synthesize` :769–789 retiene semáforo durante síntesis. [INFERENCIA] Conservar límite/config en comparación; reportar espera; no atribuir a reactor una mejora causada por cambiar admisión del proveedor

Riesgo	Medición/prueba de cierre
Caracterización H1–H8	[DOCUMENTACIÓN] Auditoría §3.5: doble finally, handlers en línea, defer30s, flags tempranos, STT huérfano, flush no-op, campo dinámico y test de texto fuente. [INFERENCIA] Trazas por escenario y lifecycle explícito; no exigir conservar fugas pero separar su corrección de la campaña de rendimiento

[INFERENCIA] La unidad de contención importa más que el promedio: publicar por llamada/shard edad de mensaje más viejo, queued_audio_ms, profundidad y pérdidas; por pool outstanding/queue wait; por reactor lag y callback más largo; por tarea CPU throttle, RSS, FDs y admisiones. Un promedio sano no cubre el peor shard ni una llamada congelada.

10. Pregunta 10 — experimento decisivo y atribución

10.1 Alcance, línea base y trazabilidad

[INFERENCIA] **Diseñar, no ejecutar todavía:** laboratorio aislado en una misma caja, sin servicios GPU reales y sin split completo. Fixtures locales de Steno/Ursula reproducen el protocolo, devuelven texto fijo por segmento y TTS grabado; un LLM stub determinista sólo permite recorrer la orquestación del monolito. No son benchmarks de STT/LLM/TTS ni certificación de negocio. Usar las interfaces de inyección existentes desde un harness; no modificar el checkout productivo para este ensayo.

[INFERENCIA] Mantener idénticos modelo/corpus/config/VNNI/ORT/secuencial/intra1/OMP1/OPENBLAS1, formatos de entrada/salida, resampling, gates, background/pacer y límites de proveedor. A/S/B comparten **el mismo core nativo** y sólo varía la asignación de dueños. Monolito usa api@08932a6 limpio mediante lectura/exportación o checkout aislado futuro, sin incorporar cambios locales. Registrar manifiesto de commit/config/binarios/dependencias/CPU/kernel/cgroup y seeds.

[CÓDIGO] El cliente api no está listo a 60 ms sólo por cambiar fps: `tools/loadtest/td1_client.py:SILENCE_FRAME/FRAME_S` :59–60 tiene 160 B/20 ms; `session` :270–280 cambia el intervalo pero usa Locutor basado en esos frames; `main` :362–373 admite barge-in y pausas, y connect:280–281 no pasa proxy-auth.

[INFERENCIA] En el laboratorio futuro, adaptar una copia del cliente a packetizar **tres tramas 160B en un WS de 480B cada 60 ms** manteniendo duración/muestras; flush parcial terminal/stall coherente con engine, sin rellenar ruido ni perder samples. Sólo poner fps=16,7 con 160B enviaría un tercio del audio. Conservar marcas, tiempos absolutos y barge-in; añadir auth para Modal. Los cambios locales actuales no son base verificada de esta comparación.

[CÓDIGO] El coalescer real depende de frame_ms efectivo=60, formato μ-law y proveedor distinto de OpenAI Realtime, y flush de stall se comprueba al entrar payload nuevo: ENG: `_send_or_coalesce_provider_audio` :5782–5845. [INFERENCIA] No suponer que existe un timer que siempre vacía a los 120 ms sin nueva llegada; reproducir ese comportamiento y terminal flush, incluidas entradas parciales.

10.2 Fixtures mínimas

Fixture	Comportamiento diseñado [INFERENCIA]
Steno	Handshake/metadata, PCM16 streaming, KeepAlive/Finalize/CloseStream, Results interinos/finales; texto con marca por sesión; latencia fija; no inferencia
Ursula	Texto con forma real aceptada por GB, PCM al sample_rate nativo, chunks y complete; audio grabado, síntesis más rápida que realtime para ejercer queue/pacer
Proveedor lento	Pausar lectura de Steno, retardar Finalize, cortar WS Ursula, chunks tardíos y síntesis fragmentada; una falla por campaña
Respuesta fija	LLM stub devuelve frases fijadas para disparar síntesis y barge-in; ningún modelo conversacional cargado
Cliente	Corpus con habla/silencio/marcas distintas por llamada y TTS superpuesto; mismos samples y calendario entre variantes

[INFERENCIA] A y S no esperan bloqueando ni un handshake ni Finalize ni síntesis. Pueden mantener la barrera lógica de entrada STT por turno mientras reciben/bufferean con cota y atienden pacer/cancel. SmartTurn fuera del dueño implica una elección explícita: si llegan nuevos frames, invalidar resultado obsoleto o retener avance lógico compatible con baseline; verificar ambas rutas con trazas. No “commit cuando termine” sin revisión.

10.3 Matriz y campañas

[INFERENCIA] Orden: (M) monolito a 60 ms → (P) Python dueño por llamada → (A) nativo dueño por llamada → (S1) nativo un shard → (S2/S4/…W) → (B) si contrato de socket resuelto. No redimensionar hardware entre campañas; fijar reserva y leerla efectiva. CPU adicional

del generador/fixtures se identifica aparte; si compite, reservar recursos de prueba fuera del presupuesto de servicio o usar otro host generador igual para todos, sin reabrir afinidad como solución de producción.

[INFERENCIA] Escalera **10 → 25 → 50 → 75 → 100** llamadas vivas, más barrido fino alrededor del primer incumplimiento; rampa de 60 s, warm-up de 60 s descartado y **10 min estables** por escalón como mínimo. Tres repeticiones independientes y orden aleatorizado/balanceado entre variantes. Cerrar/renacer procesos entre réplicas para detectar drift de memoria. N=500 es stress de pilas/FD/cola, no compromiso de capacidad.

[INFERENCIA] Campañas separadas: natural con offsets/pausas de 8 s y barge-in en un tercio de sesiones a 1.800 ms de playback; ráfaga de cierres SmartTurn; peer lento de una llamada; pérdida/reconnect; churn y drenaje. Primera campaña de capacidad sin fallos artificiales. Debe haber interrupciones antes/durante/después del gate1,2 s para comprobar rechazo y aceptación, además de finales naturales.

10.4 Métricas con puntos de tiempo comparables

[INFERENCIA] Reloj monotónico del host para duraciones; wall UTC sólo correlación. Para clocks de hosts distintos medir RTT/offset o usar observadores en el mismo host; no restar timestamps sin calibración.

Métrica	Puntos y desglose propuestos [INFERENCIA]
Latencia mensaje→VAD	Timestamp inmediatamente tras mensaje WS completo en servidor → decisión Silero del último chunk procesable; separar residencia antes de DSP, DSP y post. Mensajes sin chunk nuevo se cuentan aparte
Chunk Silero	Índice de muestra e input window; muestra acumulada aún incompleta no se etiqueta como inferencia. Latencia acústica incluye coalescer y ventana32ms por separado
SmartTurn	Trigger/snapshot → enqueue → start features → Run start/end → completion → commit; resultado obsoleto contabilizado, no considerado éxito
Barge-in local	Decisión de **gates válidos** → cancel visible en writer → último write viejo completado. Distinguir write iniciado antes de cancel, deadline y callback de completion
Barge-in cliente	Habla capturada → último audio viejo recibido; en validación con engine añadir último AudioSocket/audio oído. No equivalente a corte del envío
Pacer	Deadline→inicio/write-complete, emitted_bytes/ms, lead efectivo, underruns y ráfagas; todos los paths de audio, incluido filler/greeting
Colas	Profundidad/bytes/audio-ms/edad por llamada, carry, pool y colas IPC; TCP backlog observable; máximos y tendencia en régimen
Recursos	CPU total/por tid y por segundo de audio/turno, runtime vs runqueue wait, threads/runnable, RSS/PSS/VMS, FD, allocations y throttling
Generador	Intended_send→actual_send, samples enviados, lag loop/ping, bloqueos en send; stats separadas de servicio y fixtures
Calidad E2E local	Turnos esperados/completos, orden, transcript con cabeza de interrupción, cero mezcla, primer audio con stub y respuesta TTS completa
Agregación	p50/p90/p99 por N y por banda (10–25/26–50/51–75/76–100), por sesión/shard, TTS ON/OFF y job SmartTurn ON/OFF; histograma crudo, no percentil de percentiles

[INFERENCIA] Instrumentar monolito también desde read/enqueue: cycle_probe no incluye toda su cola. Es preciso medir la decisión VAD interna **y** cuándo el post puede actuar: el monolito espera SmartTurn antes de devolver process_audio, mientras A podría procesar decisiones frecuentes sin esperarlo. Reportar ambos, sin esconder tramas de cierre en otra categoría para aparentar p99 saludable.

10.5 Criterios falsables preregistrados

[INFERENCIA] Estos valores son **SLO experimentales propuestos**, no mediciones ni extrapolación de los 4,2 ms de cycle histórico a 20 ms. Se fijan antes de mirar resultados.

Criterio	Aceptación/rechazo diseñado [INFERENCIA]
Corrida válida	Generador intended→actual p99<=5 ms y cero samples perdidos; si no cumple, corrida inválida. Fixtures sin saturación ni cola creciente no intencional
Paridad	Todo §6 verde, cero mezcla y cero decisiones/turnos duplicados; cero pérdida de audio en régimen normal. Cualquier fallo rechaza esa implementación independientemente de velocidad
Meta N=100	VAD desde mensaje completo p90<=5 ms, p99<=20 ms; mismas cotas para peers sanos bajo fallo de uno, evaluadas aparte. Datos de SmartTurn y post reportados completos
Cancel local	Gates→último write viejo completado p99<=30 ms; cero writes viejos **iniciados** después de confirmación de corte; writes en vuelo limitados explícitamente. No usar este criterio para prometer audio oído<=30 ms
Pacer	Deadline lag p99<=20 ms, sin underrun adicional frente a baseline y sin catch-up que exceda presupuesto de lead
Backlog	Residencia de audio p99<=120 ms en normal, sin pendiente positiva del backlog en los últimos 5 min; colas dentro de cotas fijadas, overflow normal=0
Cola SmartTurn	Trigger→commit p99<=250 ms como meta separada, sin crecimiento sostenido y sin resultados obsoletos que cierren turno nuevo; prueba de ráfaga reporta máximo y tiempo de recuperación
E2E	Cero turnos perdidos; primer audio y cierre de turno con fixtures no empeoran p90 más de 10% o 20 ms (usar el mayor margen); corte recibido no empeora >20 ms vs M equivalente
Beneficio frente a M	Si M ya cumple N=100 a 60 ms, no declarar reescritura necesaria por capacidad. Para adoptar por rendimiento: aumentar K sostenible observado o reducir p99 E2E local >=20% en escalón exigente, sin regresiones anteriores
Fallos/drenaje	Cero UAF/zombies/FD residuales; cerrar admisión antes del drain; ninguna resurrección de epoch; documentar pérdida de llamadas cuando el proceso muere abruptamente

[INFERENCIA] Rechazo causal: si A no mejora la pendiente/residencia frente a M y la cola se concentra ahora en pool/lock/scheduler, el diseño trasladó el cuello en el rango probado. Si mejora VAD pero pierde turnos o empeora playback/post, se rechaza la supuesta ganancia. Si S W=1 incumple por CPU ocupado en Silero mientras A cumple, la mejora A se atribuye inicialmente a repartir ejecución; no a eficiencia automática del lenguaje.

[INFERENCIA] K_worker es el mayor N que cumple **todos** los criterios de régimen en las tres repeticiones; publicar también intervalos y casos peores. El escalón100 sólo establece K>=100 bajo esa carga si pasa; no demuestra techo. Fallos artificiales tienen informe/SLO aparte y no se mezclan con el percentil normal.

10.6 Eficiencia, paralelismo y producción

[INFERENCIA] **S W=1 es imprescindible pero insuficiente por sí solo**. Si S1 mejora a M con CPU por segundo de audio menor y misma semántica, hay ganancia de eficiencia del stack nativo sin añadir dueños paralelos; puede venir de menos Python, copias, allocations o executor. Si A mejora más a igual presupuesto pero conserva CPU/audio, la diferencia es paralelismo y aislamiento del scheduling. Si S1 y M tienen rodilla parecida y A/SW crecen con núcleos, lo observado es sobre todo distribución.

[INFERENCIA] A contra S1 no aísla por sí sola el efecto “quitar submit/resume”: ambos ya lo quitaron. Para atribuir ese mecanismo con rigor, añadir una ablación controlada en el harness: mismo core nativo/S1 con viaje a pool VAD4 frente a Silero en línea, sin cambiar buffers/modelos/pacer. También comparar P con M conservando el DSP Python. Medir handoffs efectivos y costo CPU; no confundir throughput por usar más núcleos con menor trabajo por trama.

[PENDIENTE] El laboratorio decide qué prototipo merece integración, **no** certifica K_Modal ni TTFAB real. Después, smoke Modal con wrapper/gates correctos y prueba con engine/Asterisk reales: TLS pública, proveedores reales, corte en los tres puntos y medición TTFAB. 558/1.330 ms son contexto previo, no baseline contemporáneo de fixtures. La arquitectura fija .143/Modal se conserva.

11. Pregunta 11 — esfuerzo, supuestos y orden

[INFERENCIA] Rangos en días-persona efectivos para un ingeniero con experiencia C++/Linux/voz y conocimiento de GB, modelos/corpus accesibles, caja VNNI disponible y protocolos Steno/Ursula utilizables. No incluyen esperas de permisos de proveedor, infraestructura ni campañas de producción. No se suman como si todas las fases fueran obligatorias antes de decidir: hay gates de parada.

Fase	Esfuerzo [INFERENCIA]	Riesgo dominante y salida
Baseline60ms, cliente/ledger y configuración efectiva	3–5	Confusores de generador/config; M comparable y trazas
PoC P Python dueño por llamada	5–8	Objetos ligados a loops, GC y handoff socket; decidir si necesita port
Core C++ DSP/ORT/soxr + gates de paridad y build	12–20	FFT/normalización/estado recurrente; gate ejecutable y fallo ruidoso
Laboratorio A + simuladores/probes	7–11	Red asíncrona, Finalize y generación; A funcional sin split completo
S compartiendo core + B condicionado	8–14	Fairness en S y stream bidireccional/lifetime en B; contrastes realmente comparables
Campañas/repeticiones/análisis causal	3–5	Atribución y muestras de p99; dictamen medido de K local
Frontera real C++↔Python y cinco rutas de habla	15–25	Historia/cancelación/funciones, orden y tardíos; C1–C7 y trazas
Empaquetado/supervisor/gates/Modal	4–7	Señales/readiness/ORT efectivo; humo y cold start descompuesto
Fallos, paridad E2E, ai-engine y rollout de laboratorio	6–10	Corte audible, reconnect y cleanup; aceptación E2E con estrategia de reversión
Total si se recorren todas	**63–105**	No es coste ya incurrido ni compromiso de calendario

[INFERENCIA] La estimación no presupone que todo ai-engine se arregla en los 6–10 días: si exige protocolo de generación de audio/reconexión con recuperación de estado, estimarlo por separado al conocer su alcance. Tampoco la PoC nativa debe convertirse silenciosamente en implementación del split; la integración espera la decisión experimental.

12. Lo que haría distinto y respuesta final

Cambio al planteamiento	Evidencia y consecuencia
Medir M60 y P antes de decidir reescritura	[CÓDIGO] DSP síncrono ya existe, SMART:1589–1882; [PENDIENTE] rodilla60 y GC con ese diseño. [INFERENCIA] Evita un port no justificado
Formular A como reactor async con DSP inline	[CÓDIGO] WS: `_audio_post_vad` :3780–3886 puede reconectar antes de cancel; STENO: `send_finalize` :627–720 espera resultados. [INFERENCIA] Una traducción bloqueante daña el pacer
Verificar Run concurrente antes de replicar modelos	[DOCUMENTACIÓN] ORT1.24.4 InferenceSession::Run autoriza concurrencia. [CÓDIGO] SMART:205–225 sólo lockea Run. [INFERENCIA] “43% lock” requiere medir su duración efectiva
Rechazar ORT_NO_EXCEPTIONS como aislamiento	[DOCUMENTACIÓN] Header C++1.24.4:37–73 aborta por defecto. [INFERENCIA] Error recuperable por llamada requiere status/catch
Resolver B antes de contar cero handoffs	[DOCUMENTACIÓN] Stream Beast no thread-safe; [INFERENCIA] dueño de stream o executor serializado cambia su tabla
Corregir gate y certificar el binario nativo	[CÓDIGO] arranque.sh:43–44 sólo observa pipeline; [INFERENCIA] gate Python/CPU no cubre otra lib/model runner
Resolver prioridad/seq y barrera STT	[DOCUMENTACIÓN] C1/C7 de matriz; [CÓDIGO] STENO:836–848 presume receive bloqueado. [INFERENCIA] La frontera nueva necesita orden wire y segmentación explícitos
No llamar played a bytes enviados	[CÓDIGO] WS: `_TTSPacer.played_bytes` :401–408 estima por wall time; AS: `send_audio` :263–267 sólo confirma write/drain. [INFERENCIA] emitted_ms no prueba lo oído
Separar eficiencia de paralelismo	[INFERENCIA] S1/M/P y ablación executor del §10 evitan atribuir a C++ una ganancia obtenida sólo por distribuir llamadas

[INFERENCIA] **Respuesta:** un MediaWorker C++ con dueño por llamada **elimina el mecanismo concreto de coordinación global submit/resume VAD**, siempre que reciba directamente, procese Silero en su dueño y no introduzca un relay ni esperas bloqueantes en la ruta frecuente. No elimina todo cuello ni acredita capacidad: los costos de DSP/post permanecen y puede trasladar el límite al scheduler, pool SmartTurn, ORT o transporte. **Primero probar A**, con SmartTurn fuera y reactor no bloqueante; acompañarlo de S W=1 y de baseline/Python60. B viene después de demostrar ownership del WebSocket y necesidad de aislar direcciones. La decisión de producción la cierra el experimento con paridad y E2E, no la analogía con FreeSWITCH ni el lenguaje.

[DOCUMENTACIÓN] Este encargo sólo produce el dictamen. No se implementó el port, el laboratorio ni cambios en GoldenBridge, ai-engine o repositorios auditados; no se hicieron commits.